ECSE-6320 Advanced Computer Systems
Project 2 Report

Cooper Werner
Rensselaer Polytechnic Institute
wernec6@rpi.edu
RIN: 662058426

September 29, 2026

Abstract

Contents

1 Introduction **2**

2 Pre-Experiment Hypotheses **2**

3 Correctness Validation **3**
3.1 Validation Method 3
3.2 Validation Results 3

4 Measurement Discipline **4**
4.1 Experimental Setup 4
4.2 Statistical Treatment 4

5 Results **4**
5.1 Summary Figure 4
5.2 Secondary Figures 4
5.3 Results Table 4
5.4 Relevant Code 4

6 Negative / Failure Case **5**

7 Honest Anomalies **5**

8 Discussion **5**

9 Conclusion **5**

A Additional Data / Raw Logs **5**

1 Introduction

The goal of this project is to analyze how the memory hierarchy impacts performance of the simple computations as the working set size, data stride & access pattern, read-write ratio, access intensity, and page size changes.

To do this I will use 4 different computational kernels:

1. Array Reduction, Summing all values of an array: 100% read.
2. Memset, setting all values in an array to one value: 100% write.
3. Vector-Vector Multiply, element-wise multiplication of two vectors: 66% read / 33% write.
4. Scalar-Vector Multiply, element-wise multiplication with a scalar: 50 % read / 50% write.

To change the other experimental variables, I have a main kernel runner that can control the working set size, the stride of the data, run with random access patterns, number of threads accessing the data. To be able to control the page size, I will be running this code in a Linux VM where I can set the memory page size to either 4KiB or 2MiB.

2 Pre-Experiment Hypotheses

Pre-Experiment Hypothesis

Hypothesis:

Huge pages will help with reducing the number of TLB misses. However, this will only help when the working set size is large, for small working sets that fit into a small number of pages, the larger page sizes will have little if any impact on performance.

Reasoning:

For larger working set sizes, with small pages of 4KiB, the data will span potentially thousands of pages creating a large number of TLB misses, each TLB miss causing a page fault and requiring a ring level switch to fetch a page address. This ring level switch will become a large part of the the time spent during computation.

By switching to large pages, the larger working set data can fit into a fewer number of pages, reducing the number of TLB cache misses and therefore reducing the time spent handling page faults, improving performance.

Predicted quantitative outcome:

Table 1: Huge Page Speedup

Working Set Size	Huge Page Speedup
1,024	$1x$
16,384	$1.05x$
65,536	$1.1x$
1,048,576	$1.5x$
16,777,216	$2x$
268,435,456	$2.7x$

Pre-Experiment Hypothesis

Hypothesis:

As the access pattern demonstrates lower spatial locality with larger strides and random access patterns, the computational performance will drop significantly as there will be more cache misses.

Reasoning:

The more sporadic access patterns will have to fetch from main memory more frequently, this will result in more pipeline stalls as data is no longer resident in cache. With the smaller strides, more useful data will be in each cache line and therefore more data will be able to be prefetched.

Predicted quantitative outcome:

Table 2: Predicted SIMD Slowdown relative to SIMD with 0 byte offset

Stride Size (Elements)	**Slowdown**
1	$1x$
2	$0.99x$
4	$0.95x$
8	$0.87x$
16	$0.79x$
32	$0.70x$
64	$0.65x$
128	$0.45x$
256	$0.39x$
Random	$0.25x$

3 Correctness Validation

3.1 Validation Method

3.2 Validation Results

Table 3: Correctness validation summary

Check	Expected	Observed

4 Measurement Discipline

4.1 Experimental Setup

- **Hardware:** CPU: x86_64 AMD Ryzen AI 7 350 w/ Radeon 860M (SIMD Support: SSE, SSE2, AVX, AVX2, AVX-512), Memory: 32GB LPDDR5 @ 5600MT/s Virtualized with VMWare Workstation, in a VM with 8 CPU cores and 16GB of RAM.
- **OS / Kernel:** Pop_OS! 24.04 Running Linux 7.0.11-76070011-generic running in VMWare Workstation
- **Compiler & Flags:** GCC 16.2.0, Flags: -O3
- **Isolation techniques:** Ran on CPU0 with 100% affinity, and -20 niceness (ensure no context switching)
- **Trials per condition:** 10 trials per condition
- **Timer / Instrumentation:** clock_gettime(CLOCK_MONOTONIC), perf for measuring TLB and memory events, Intel Memory Latency Checker

4.2 Statistical Treatment

5 Results

5.1 Summary Figure

Figure 1: Caption describing the primary result.

5.2 Secondary Figures

(a) (b)

Figure 2: Side-by-side comparison of . . .

5.3 Results Table

Table 4: Quantitative results

Configuration	Mean	Std. Dev.	N

5.4 Relevant Code

```
int main(void) {
    // ...
    return 0;
```

```
}
```

Listing 1: Benchmark kernel (C)

```cpp
#include <chrono>

int main() { auto start = std::chrono::high_resolution_clock::now(); // ... }
```

Listing 2: Measurement harness (C++)

```rust
fn main() {
    let start = std::time::Instant::now();
    // ...
}
```

Listing 3: Benchmark harness (Rust)

6 Negative / Failure Case

Negative / Failure Case

What failed:

Evidence:

Root cause (if known):

7 Honest Anomalies

Honest Anomalies

Anomaly 1:

Anomaly 2:

8 Discussion

9 Conclusion

A Additional Data / Raw Logs